

Proyecto II
Documento Tecnico

Joao Arauz Rodriguez
Abraham Rojas Duarte

EIF 204
Programacion II

Jonathan Morales Murillo

Universidad Nacional
Escuela de Informatica

1. ¿Qué es?

Soul Knight es una simulación de aventuras en consola hecha en C++. El usuario elige un personaje, explora 3 niveles de mazmorras con enemigos aleatorios y pelea contra jefes al final de cada nivel. Todo se ejecuta en terminal.

2. Archivos de entrada

Archivo	Formato	Ejemplo
characters.txt	nombre salud ataque	Knight 120 15
enemies.txt	nombre salud ataque	Goblin 25 5
events.txt	`nombre	descripcion
items.txt	`tipo	nombre

Nota: enemigos con salud ≥ 80 se consideran jefes automáticamente.

3. Estructura de clases

Entity (abstracta)
├── Characters (jugador)
└── Enemy (enemigo)

Room (abstracta)
├── SimpleRoom
└── BossRoom

Ability (abstracta)
├── KnightAbility
├── WizardAbility
├── RogueAbility
└── PriestAbility

Relaciones principales:

Characters tiene una Ability y un Inventory

Level contiene varias Room*

GameEngine coordina todo: niveles, jugador, combate, eventos, items y logger

CombatSystem resuelve peleas entre entidades

CharacterManager y EnemyManager cargan datos desde archivos

4. Decisiones de diseño

Decisión	Por qué
Strategy para habilidades	Cada personaje tiene comportamiento distinto en combate sin tocar la lógica del CombatSystem
Template en Item<T>	Permite items con distintos tipos de valor (int para armas/pociones, extensible a otros)
Iterador en Inventory	Se puede recorrer el inventario con range-based for

Decisión	Por qué
Polimorfismo en Room	Se pueden agregar nuevos tipos de sala sin cambiar Level
EventSystem desde archivo	Los eventos se cargan externamente, fácil de modificar sin recompilar

5. Manejo de memoria

Las salas (Room*) se liberan en el destructor de Level

Las habilidades (Ability*) se liberan en el destructor de Characters

Los niveles se liberan en el destructor de GameEngine

Se evitan copias innecesarias usando referencias

6. Manejo de errores

Se valida apertura de archivos antes de leer

Se validan datos leídos (valores positivos, formatos correctos)

Se usan excepciones (runtime_error, invalid_argument) con try/catch en main

Se valida entrada del usuario con cin.fail() y recuperación

7. Técnicas del curso aplicadas

Técnica	Dónde se usa
Herencia	Entity→Characters/Enemy, Room→SimpleRoom/BossRoom
Polimorfismo	Room*, Ability* con métodos virtuales
Clases abstractas	Entity, Room, Ability
Templates	Item<T>
Sobrecarga de operadores	==, <, >, +=, -=, <<
Manejo de archivos	ifstream/ofstream en managers y logger
Excepciones	try/catch con tipos específicos
Patrón Strategy	Ability y sus implementaciones
Patrón Factory	CharacterManager, EnemyManager
STL	vector, string

8. Archivos que genera

log.txt — bitácora de todo lo que pasó en la partida

report.txt — resumen final con estadísticas

9. Limitaciones

El recorrido es lineal (no hay bifurcaciones ni elección de camino).

No hay sistema de guardado/carga de partida.

Los artículos se usan inmediatamente en lugar de almacenarse para uso posterior.

La interfaz es puramente textual.

Posibles mejoras

Exploración no lineal.

Agregar sistema de guardado usando serialización.

Implementar un inventario activo donde el jugador elija cuándo usar artículos.

Agregar más tipos de sala (tienda, trampa, rompecabezas).

Implementar punteros inteligentes (unique_ptr) para gestión automática de memoria.

Agregar sistema de logros/achievements.